

20 PAGES
PRODUCTION FOCUS
SENIOR ENGINEER
QUICK REFERENCE

DEVOPS POCKET GUIDE

A 20-PAGE PRACTICAL REFERENCE FOR
BUILDING, DEPLOYING, OPERATING AND SCALING
MODERN SYSTEMS



Linux



Git



Jenkins



Docker



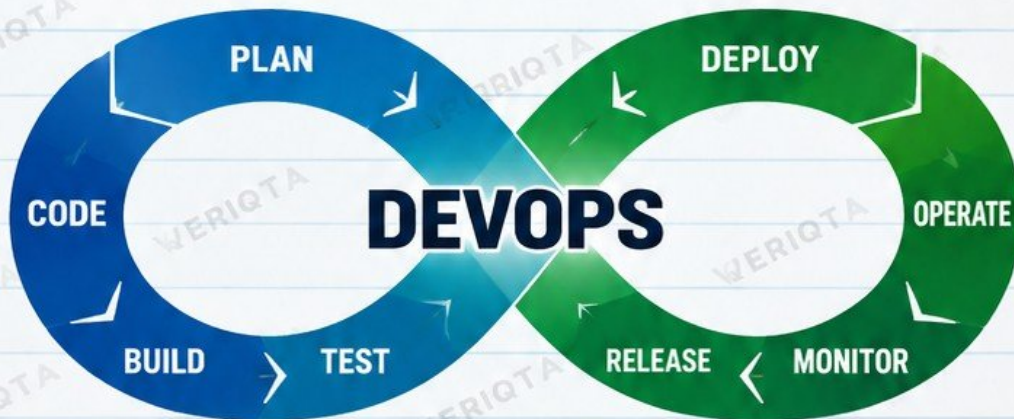
Kubernetes



Terraform



AWS



REAL-WORLD
EXAMPLES



PRACTICAL
COMMANDS



TROUBLESHOOTING
INSIGHTS



PRODUCTION
BEST PRACTICES

SMALL GUIDE. BIG IMPACT.

DevOps Production Mental Model

From Commit to Customer – Build, Deploy, Operate, Improve

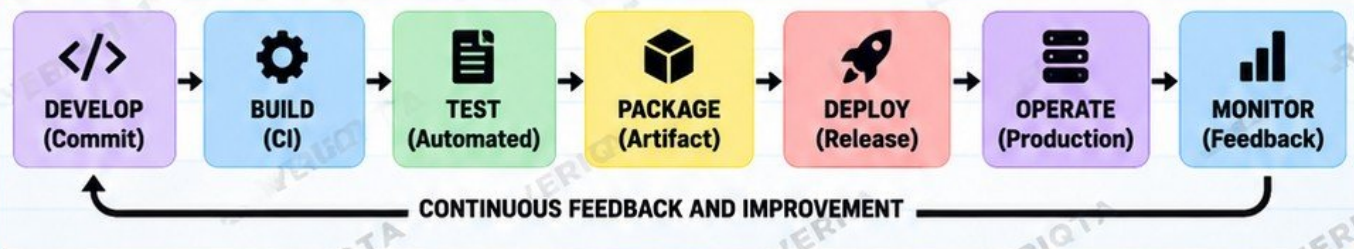
1. WHAT IS THE DEVOPS MENTAL MODEL?

- DevOps is a continuous flow of value from development to production with shared ownership.
- It combines people, process, automation and monitoring to deliver reliable software at scale.
- It is not a tool. It is a way of thinking, designing and operating systems.

2. WHY IT MATTERS

- Faster and safer delivery
- Higher reliability and availability
- Better collaboration between dev, ops, security and business
- Lower operational risk and cost
- Continuous learning and improvement
- Happier customers

3. FLOW FROM COMMIT TO PRODUCTION



4. KEY PRINCIPLES

- Shared ownership (you build it, you run it)
- Automation over manual work
- Small, frequent and safe changes
- Observability from day one
- Security built in (DevSecOps)
- Measure, learn and improve
- Focus on customer value

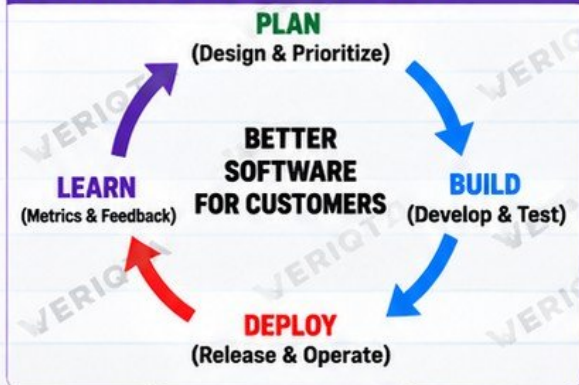
5. FAILURE DOMAINS

- Code and application bugs
- Infrastructure and platform
- Dependencies (databases, APIs, third-party services)
- Network and DNS
- Configuration and secrets
- Human error
- Cloud or region outages
- Security incidents

6. OPERATIONAL RESPONSIBILITY

- Ensure systems are reliable, scalable and secure
- Monitor, respond and recover from incidents
- Keep runbooks and documentation updated
- Plan for capacity and cost
- Continuously reduce toil
- Work with product to meet user needs and SLAs

7. THE DEVOPS FEEDBACK LOOP



8. SENIOR ENGINEER THINKING

- Think in systems, not isolated components
- Understand trade-offs (speed vs stability, cost vs resilience)
- Design for failure and recovery
- Reduce blast radius
- Make observability a first-class requirement
- Automate, standardize and document
- Continuously improve through real production experience

9. REAL WORLD LESSON



"In production, everything works... until it doesn't.
Good DevOps engineers design for the day it breaks."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Linux Production Survival Guide

Key Areas, Commands and What to Look For in Production

1. CPU & LOAD



- Check CPU usage, load average and running processes.
- High load can mean CPU saturation, I/O wait or blocked processes.

```
top
htop
Uptime
mpstat -P ALL 1
sar -u 1 5
```

2. MEMORY



- Check memory usage, swap and memory pressure.
- Look for OOM kills and processes consuming excessive memory.

```
free -h
vmstat 1 5
cat /proc/meminfo
dmesg | grep -i oom
ps aux --sort=-%mem | head
```

3. DISK I/O & STORAGE



- Check disk usage, I/O performance and inode usage.
- High I/O wait can slow down applications even if CPU is low.

```
df -h
du -sh /path
iostat -x 1 5
lsblk
find / -xdev -type f -size +1G
```

4. PROCESSES



- Find what is running, resource usage and process hierarchy.
- Identify runaway or zombie processes.

```
ps aux
ps -ef --forest
top -o %CPU
pgrep -a <name>
kill -TERM <pid>
```

5. FILE DESCRIPTORS



- Check open files and limits.
- High file descriptor usage can cause service failures (e.g. "Too many open files").

```
ulimit -n
cat /proc/sys/fs/file-max
lsdf | wc -l
lsdf -p <pid> | wc -l
cat /proc/<pid>/limits
```

6. SOCKETS & NETWORK CONNECTIONS



- Check listening ports, active connections and states.
- Useful for debugging service connectivity and unexpected traffic.

```
ss -tulnp
ss -s
ss -tan | head
lsdf -i -P -n
netstat -s
```

7. SYSTEMD



- Manage and troubleshoot system services.
- Check service status, logs and failures.

```
systemctl status <service>
systemctl restart <service>
systemctl enable <service>
systemctl list-units --failed
systemctl daemon-reload
```

8. JOURNALD (LOGS)



- View system and service logs with journald.
- Filter by service, time or severity.

```
journalctl -u <service>
journalctl -xe
journalctl --since "1 hour ago"
journalctl -p err
journalctl -k (kernel logs)
```

9. KERNEL EVIDENCE



- Check kernel messages for hardware, filesystem and kernel issues.
- Look for errors, warnings and driver problems.

```
dmesg | less
dmesg | grep -i error
dmesg | grep -i warning
cat /var/log/kern.log
uname -a
```

10. PRODUCTION TIPS



- Set up monitoring and alerting.
- Know normal baseline for your systems.
- Investigate before making changes.
- Keep logs, metrics and runbooks.
- Automate repetitive tasks.

COMMON WARNING SIGNS



- High CPU or load average
- Memory pressure or OOM kills
- Disk space or inode exhaustion
- Too many open files
- Service crashes or restarts
- Kernel errors or hardware issues

REMEMBER



"A healthy Linux system is observable, predictable and automated."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Linux Incident Command Toolkit

Essential Commands for Fast, Evidence-First Investigation

1. top (Real-time processes)



- View live CPU, memory and process activity.
- Identify high resource consumers.

```
top
top -o %CPU
top -o %MEM
```

2. ps (Process snapshot)



- Check running processes.
- Filter and find specific processes.

```
ps aux
ps -ef | grep <name>
ps -eo pid,ppid,cmd,%cpu,%mem
ps -ef --forest
```

3. ss (Sockets & connections)



- View network connections, listening ports and states.
- Useful for debugging connectivity issues.

```
ss -tulnp
ss -s
ss -tan | head
ss -p | grep :<port>
```

4. lsof (Open files)



- See what files, sockets and ports are in use.
- Find which process is using a file or port.

```
lsof -nP
lsof -i :<port>
lsof /path/to/file
lsof | grep <name>
```

5. df (Disk space)



- Check filesystem usage and available space.
- Identify full partitions quickly.

```
df -h
df -i (inode usage)
df -Th
df --total
```

6. du (Directory usage)



- Find large directories and files.
- Useful for investigating disk space issues.

```
du -sh /path
du -h --max-depth=1 /path
du -ah | sort -rh | head
du -sh *
```

7. free (Memory usage)



- Check memory usage, free memory and swap.
- Identify memory pressure and potential OOM issues.

```
free -h
free -m
free -s 5
cat /proc/meminfo
```

8. vmstat (System overview)



- View CPU, memory, I/O and system activity.
- Useful for spotting bottlenecks.

```
vmstat 1 5
vmstat -a 1 5
vmstat -d 1 5
vmstat -p 1 5
```

9. iostat (I/O performance)



- Check disk I/O, throughput and utilization.
- Identify I/O wait and disk bottlenecks.

```
iostat -x 1 5
iostat -d 1 5
iostat -m 1 5
iostat -xz 1 5
```

10. journalctl (System logs)



- View and filter systemd logs.
- Investigate service failures and recent events.

```
journalctl -xe
journalctl -u <service>
journalctl --since "1 hour ago"
journalctl -k (kernel logs)
```

11. dmesg (Kernel messages)



- View kernel ring buffer messages.
- Check for hardware, filesystem and driver issues.

```
dmesg | tail
dmesg | grep -i error
dmesg | grep -i warning
dmesg -T (show timestamps)
```

12. EVIDENCE-FIRST INVESTIGATION



- 1 Define the symptoms
- 2 Check recent changes
- 3 Gather evidence (metrics, logs)
- 4 Form hypotheses
- 5 Test and narrow the cause
- 6 Fix, verify and prevent recurrence

PRODUCTION TIPS



- Know your normal baseline.
- Use multiple commands, not just one.
- Correlate metrics, logs and recent changes.
- Document findings and fixes.

COMMON MISTAKES



- Blaming the wrong component.
- Ignoring logs and kernel messages.
- Making changes without evidence.
- Not checking recent deployments.
- Forgetting to verify the fix.

REMEMBER



"In production, facts come from evidence, not assumptions."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



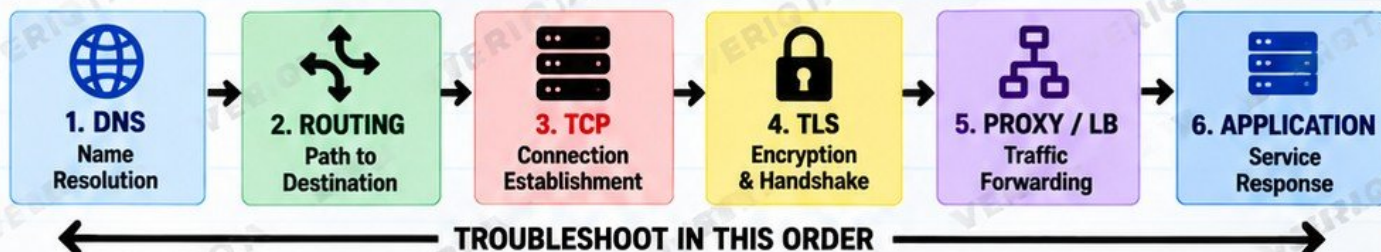
@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Networking Troubleshooting Map

DNS → Routing → TCP → TLS → Proxy/LB → Application

A structured approach to quickly find and fix network issues in production.



1. DNS TROUBLESHOOTING

- Check name resolution
- Verify correct IP address
- Check DNS server and TTL
- Identify DNS latency or failures

```
nslookup example.com
dig example.com
dig +trace example.com
host example.com
```

2. ROUTING TROUBLESHOOTING

- Check network path
- Verify hops and latency
- Identify routing issues
- Check for network policies or firewalls

```
ip route
traceroute example.com
mtr example.com
curl --connect-timeout 5 \
example.com
```

3. TCP TROUBLESHOOTING

- Check if port is open
- Verify connection establishment
- Look for connection timeouts
- Check for packet loss or resets

```
nc -zv example.com 80
telnet example.com 80
ss -tan | grep :80
tcpdump -nn host example.com
```

4. TLS TROUBLESHOOTING

- Check certificate validity
- Verify TLS handshake
- Identify protocol or cipher issues
- Check certificate chain

```
openssl s_client -connect \
example.com:443
openssl x509 -noout -text -in cert.pem
curl -v https://example.com
nmap --script ssl-enum-ciphers \
-p 443 example.com
```

5. PROXY / LOAD BALANCER

- Check if traffic reaches the LB
- Verify health checks
- Inspect proxy headers
- Identify misrouting or misconfiguration

```
curl -I http://example.com
curl -H "Host: example.com" \
http://<lb-ip>
nginx -T (check config)
ss -tan | grep :443
```

6. APPLICATION LAYER

- Verify service is running
- Check response codes and latency
- Inspect application logs
- Confirm dependency connectivity

```
curl -v http://example.com/health
curl -w "%{time_total}\n" \
http://example.com
tail -f /var/log/app.log
ss -tan | grep <app-port>
```

COMMON NETWORK ISSUES



- DNS resolution failures
- No route to host
- Connection timeouts
- TLS handshake failures
- 502 / 503 from load balancer
- High latency or packet loss
- MTU / fragmentation issues
- Firewall or security group blocks

USEFUL DIAGNOSTIC COMMANDS



- ping - Test reachability
- traceroute - Trace network path
- mtr - Real-time path analysis
- ss - Check connections
- tcpdump - Capture packets
- curl - Test HTTP/HTTPS
- openssl - Debug TLS
- dig/nslookup - DNS lookup
- ip route - Check routing
- journalctl - Check system logs

NETWORK TROUBLESHOOTING FLOW

- 1 Reproduce the issue
- 2 Identify the failure domain
- 3 Check from DNS to application
- 4 Gather evidence (commands, logs)
- 5 Form a hypothesis
- 6 Test and confirm
- 7 Fix the root cause
- 8 Verify and monitor

KEY CONCEPTS



- Packet loss affects all layers
- MTU mismatch causes fragmentation
- Firewalls can silently drop traffic
- DNS issues can look like application issues
- TLS failures are often certificate or protocol related
- Use timeouts to avoid long delays

PRODUCTION TIP



"Always verify each layer before moving to the next. Most network issues are caused by a small number of common problems."

REMEMBER



"It's not a network issue until you have evidence."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Git for Production Engineering

Branching, Safety, History, Recovery and Best Practices for Real Systems

1. BRANCHING STRATEGY



- Use a clear branching model (e.g. main, develop, feature, release, hotfix).
- Keep main stable and always deployable.
- Use short-lived feature branches.
- Use release branches for controlled releases.
- Use hotfix branches for urgent fixes.

```
git checkout -b feature/login
git checkout -b release/v1.2.0
git checkout -b hotfix/critical-fix
```

2. PROTECTED BRANCHES



- Protect main and release branches.
- Require pull requests (PRs).
- Enforce code reviews.
- Enable status checks (CI).
- Block force pushes.
- Restrict who can merge.

EXAMPLE (GitHub)

- Require PR reviews
- Require passing CI checks
- Include administrators
- Restrict force pushes
- Require signed commits (optional)

3. COMMIT INTEGRITY



- Write clear, meaningful commit messages.
- Keep commits small and focused.
- Use signed commits for security.
- Link commits to issues.
- Avoid committing secrets, large files or generated artifacts.

```
git commit -m "Add login endpoint (fixes #123)"
```

```
git commit -S -m "Signed commit"
```

```
git log --oneline --graph --decorate
```

4. REVERT vs RESET



- Use git revert to safely undo changes in shared branches.
- Use git reset only in local or unshared branches.
- Revert keeps history (safe for production).
- Reset rewrites history (dangerous in shared repos).

```
# Safe (keeps history)
git revert <commit-id>
```

```
# Dangerous (use only locally)
git reset --hard <commit-id>
```

5. TAGS (RELEASES)



- Use tags to mark release versions.
- Use semantic versioning (e.g. v1.2.0).
- Tags make it easy to rollback.
- Use annotated tags for releases.

```
# Create annotated tag
git tag -a v1.2.0 -m "Release 1.2.0"
# Push tag
git push origin v1.2.0
# List tags
git tag -l
# Checkout a tag
git checkout v1.2.0
```

6. CONFLICT RISK & RESOLUTION



- Conflicts happen when multiple people change the same lines.
- Pull frequently to reduce risk.
- Use clear file ownership.
- Resolve conflicts carefully and test before merging.

```
# Update your branch
git pull origin main
# Resolve conflicts (edit files)
git status
# After resolving
git add <file>
git commit -m "Resolve conflict"
```

7. RECOVERY SCENARIOS



- Recover deleted branches.
- Find lost commits.
- Undo bad merges.
- Use reflog when things go wrong.
- Keep backups (remote repos).

```
# Recover deleted branch
git reflog
git checkout -b recovered <commit-id>
# Find a lost commit
git fsck --lost-found
```

8. PRODUCTION GIT BEST PRACTICES



- Keep main always deployable.
- Use PRs, reviews and CI.
- Sign commits and tags.
- Use meaningful messages.
- Clean .gitignore.
- Avoid force pushes.
- Automate checks (lint, tests, scan).
- Document your branching strategy.
- Regularly prune old branches and tags.

9. QUICK COMMAND REFERENCE



git status	Check status
git branch	List branches
git switch <branch>	Switch branch
git pull	Update branch
git push	Push changes
git log --oneline	View history
git diff	Show changes
git merge <branch>	Merge branch
git revert <id>	Revert commit
git reset --hard <id>	Reset (use with care)
git tag -a v1.0.0	Create tag
git stash	Stash changes

KEY TAKEAWAY



"Good Git practices prevent incidents, protect history and keep your production environment safe."

REAL WORLD LESSON



"Most production problems are not caused by Git, but by poor branching discipline and lack of code reviews."

REMEMBER



"Communicate, keep history safe, and always verify before you push."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

CI/CD Pipeline Architecture

Build → Test → Scan → Package → Publish → Deploy → Verify
Artifact Promotion, Reproducibility, Isolation



← AUTOMATED • REPEATABLE • SECURE • OBSERVABLE →

1. PIPELINE OVERVIEW



- Automate the delivery process from code to production.
- Ensure consistency, security and speed.
- Reduce manual work and human error.
- Use the same pipeline for all environments (with controls).

“Same process. Different environments. Controlled promotion.”

2. BUILD STAGE



- Checkout source code.
- Install dependencies.
- Compile and build the application.
- Run basic sanity checks.

```
# Example (generic)
git clone <repo>
cd <repo>
make build
# or
mvn clean package
# or
npm ci && npm run build
```

3. TEST STAGE



- Run unit tests.
- Run integration tests.
- Check test coverage.
- Fail the pipeline if tests fail.

```
# Example
pytest -v
mvn test
npm test
# Fail on error (most CI tools do this automatically)
```

4. SCAN STAGE



- Scan for vulnerabilities (dependencies, containers, IaC).
- Check code quality and linting.
- Enforce security policies.
- Fail or warn based on risk level.

```
# Example tools
trivy image <image>
sonar-scanner
npm audit
snyk test
```

5. PACKAGE STAGE



- Package the application.
- Create versioned artifacts.
- Use immutable artifacts (e.g. Docker image, JAR, ZIP).
- Include metadata (version, commit).

```
# Example
# Docker
docker build -t app:1.0.0 .
docker tag app:1.0.0 registry/app:1.0.0
# Archive
tar -czf app-1.0.0.tar.gz ./dist
```

6. PUBLISH STAGE



- Push artifacts to a registry.
- Use artifact repository (e.g. Docker Registry, Nexus, Artifactory).
- Tag with version, commit SHA.
- Restrict access with permissions.

```
# Example (Docker)
docker push registry/app:1.0.0
# Example (Nexus)
curl -u user:pass --upload-file app.jar \
https://nexus.example.com/repository/
```

7. DEPLOY STAGE



- Deploy to target environment (dev, staging, production).
- Use IaC or deployment tools (Helm, Kubernetes, Terraform).
- Implement safe deployment strategies (rolling, blue-green, canary).

```
# Example (Kubernetes)
helm upgrade --install app ./chart \
--namespace prod --set image.tag=1.0.0
```

8. VERIFY STAGE



- Check application health.
- Run smoke tests.
- Verify metrics, logs and alerts.
- Confirm functionality and performance.

```
# Example
curl -f https://app.example.com/health
kubectl rollout status deployment/app
# or run automated tests
```

9. ARTIFACT PROMOTION



- Promote the same artifact across environments.
- Do not rebuild for each environment.
- Use signed and versioned artifacts.
- Maintain a clear promotion process (with approvals if required).

“Build once.
Promote everywhere.”

KEY PRINCIPLES



- Reproducible builds
- Immutable artifacts
- Environment isolation
- Security and compliance
- Observability and traceability

COMMON MISTAKES



- Rebuilding artifacts for each environment
- Skipping tests or security scans
- Using mutable tags (e.g. latest)
- No rollback plan
- Lack of environment isolation
- No monitoring after deployment

REMEMBER



“A reliable pipeline
is the foundation
of a reliable system.”



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Safe Deployment Strategies

Rolling, Blue-Green, Canary, Feature Flags, Progressive Delivery Rollback Criteria, Blast-Radius Control

1. ROLLING DEPLOYMENT

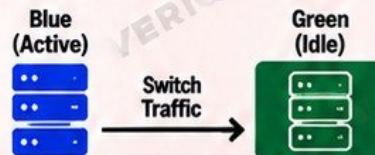


- Replace instances gradually.
- No downtime.
- Works well for stateless applications.
- Keep at least one healthy instance.
- Monitor for errors during rollout.

Example (Kubernetes)

```
kubectl set image deployment/app \
  app=v2
kubectl rollout status deployment/app
```

2. BLUE-GREEN DEPLOYMENT

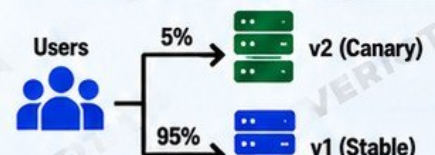


- Run two identical environments.
- Switch traffic after full validation.
- Near zero downtime.
- Easy and fast rollback.
- Requires more infrastructure (cost).

Example (concept)

1. Deploy v2 to green environment
2. Run tests
3. Switch load balancer to green
4. Keep blue as fallback

3. CANARY DEPLOYMENT



- Deploy to a small percentage of users.
- Monitor metrics, logs and errors.
- Gradually increase traffic if healthy.
- Minimizes risk and blast radius.
- Automatically rollback if issues detected.

Example (Kubernetes)

```
kubectl apply -f canary.yaml
# Increase traffic gradually
# Use service mesh or ingress (e.g. Istio)
```

4. FEATURE FLAGS



- Enable or disable features at runtime.
- Deploy code safely without exposing to all users.
- Useful for A/B testing and gradual rollouts.
- Quickly turn off problematic features.
- Decouples deployment from release.

Example (application code)

```
if (featureFlags.isEnabled("new_feature")) {
  // new code
} else {
  // old code
}
```

5. PROGRESSIVE DELIVERY



- Gradually increase exposure to new version.
- Use metrics, SLOs and user feedback.
- Combine techniques (canary, feature flags, A/B testing).
- Automate promotion based on health checks.
- Improves safety and user experience.

Key metrics to monitor

- Error rate
- Latency (p95/p99)
- Success rate
- Business metrics (signups, purchases)
- User feedback

6. ROLLBACK STRATEGIES



- Define clear rollback criteria before deployment.
- Implement automated rollback where possible.
- Keep previous version ready and tested.
- Use immutable artifacts.
- Verify system health after rollback.

Example (Kubernetes)

```
kubectl rollout undo deployment/app
kubectl rollout history deployment/app
kubectl rollout status deployment/app
```

7. BLAST-RADIUS CONTROL



- Limit the impact of failures.
- Use small, isolated deployments (e.g. per region, per tenant).
- Apply rate limits, circuit breakers and bulkheads.
- Use canary or progressive rollout.
- Monitor and set alerts.

Techniques

- Isolation (namespaces, VPCs)
- Traffic segmentation
- Circuit breakers
- Rate limiting
- Multi-region deployment

8. DEPLOYMENT DECISION CHECKLIST



- Have we tested in a staging environment?
- Are monitoring and alerts in place?
- Is rollback ready and tested?
- What is the blast radius if this fails?
- Do we have a clear success metric?
- Is the deployment automated and repeatable?
- Have we communicated the change?
- Do we have approval?

9. KEY TAKEAWAYS



- Choose the right strategy for your use case.
- Start small, measure, and increase gradually.
- Automate and be ready to rollback.
- Monitor everything.
- Control the blast radius.
- A safe deployment is a repeatable deployment.

REMEMBER



"Deploy fast, but always deploy safely."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQT
ENGINEER A
BETTER TOMORROW

Docker Production Guide

Layers, Build Cache, Multi-Stage Builds, Runtime Isolation, Signals, PID 1, Storage, Networking, Resource Limits

1. CONTAINER BASICS



- Understand what a container is.
- Containers share the host kernel.
- Use images to create containers.
- Keep images small and secure.
- Use official or trusted base images.

```
docker version
docker pull nginx
docker run -d --name web nginx
docker ps
docker logs web
```

2. LAYERS & BUILD CACHE



- Each instruction creates a layer.
- Order matters for cache efficiency.
- Put rarely changing steps first.
- Use .dockerignore to reduce build context.
- Smaller layers = faster builds.

```
docker build -t app:1.0 .
docker build --no-cache -t app:1.0 .
docker history app:1.0
docker image inspect app:1.0
```

3. MULTI-STAGE BUILDS



- Use multi-stage builds to create smaller images.
- Build dependencies in a separate stage.
- Copy only what you need into the final image.
- Keep the runtime image minimal.

```
FROM golang:1.22 AS builder
WORKDIR /app
RUN go build -o app

FROM gcr.io/distroless/base
COPY --from=builder /app /app
ENTRYPOINT ["/app"]
```

4. RUNTIME ISOLATION



- Use non-root users.
- Apply least privilege.
- Isolate with namespaces and cgroups (handled by Docker).
- Use read-only file systems where possible.
- Drop unnecessary capabilities.

```
docker run -d --user 1000:1000 \
--read-only --cap-drop=ALL \
--name app app:1.0
```

5. SIGNALS & PID 1



- PID 1 behaves differently.
- Use exec form in Dockerfile.
- Handle SIGTERM for graceful shutdown.
- Use a proper init system or tini.
- Avoid zombie processes.

```
# Use exec form
ENTRYPOINT ["/app"]

# Or use tini
docker run --init app:1.0
```

6. STORAGE



- Understand container storage drivers (overlay2).
- Use volumes for persistent data.
- Keep container filesystem stateless when possible.
- Know when to use bind mounts vs named volumes.

```
docker volume create app-data
docker run -d -v app-data:/data app
docker run -v $(pwd):/app app
```

7. NETWORKING



- Use Docker networks to isolate containers.
- Understand bridge, host, and overlay networks.
- Use service discovery by container name.
- Expose only required ports.
- Use user-defined networks.

```
docker network create app-net
docker run -d --name db --network app-net
postgres:16
docker run -d --name app --network app-net
app:1.0
```

8. RESOURCE LIMITS



- Set CPU and memory limits.
- Prevent noisy neighbor issues.
- Use resource constraints in production.
- Monitor container resource usage.
- Right-size your containers.

```
docker run -d --name app \
--cpus="0.5" --memory="512m" \
--memory-swap="1g" app:1.0

docker stats
docker update --cpus="1" --memory="1g" app
```

9. PRODUCTION BEST PRACTICES



- Use small, secure, and reproducible images.
- Scan images for vulnerabilities.
- Use multi-stage builds.
- Set resource limits.
- Run as non-root.
- Handle signals and graceful shutdown.
- Use health checks.
- Keep containers immutable.
- Use logging and monitoring.
- Follow least privilege and defense in depth.

COMMON MISTAKES



- Using latest tag in production.
- Running as root.
- Ignoring .dockerignore.
- Building large images.
- Not setting resource limits.
- Not handling signals (PID 1 issues).
- Storing data inside the container.

QUICK COMMAND REFERENCE



```
docker build -t app .
docker run -d app
docker ps
docker logs app
docker exec -it app bash
docker stop app
docker rm app
docker images
docker network ls
docker volume ls
```

REMEMBER



"Small images, secure containers, reliable deployments."

CONTAINERS ENABLE SPEED,
BUT DISCIPLINE KEEPS
YOU IN PRODUCTION.

Container Security Checklist

Minimal Images, Non-Root Execution, Capabilities, Secrets, Scanning, Signing, SBOMs, Supply-Chain Controls

1. MINIMAL & TRUSTED IMAGES



- Use minimal base images (e.g. distroless, alpine).
- Use official and trusted sources.
- Keep images small (fewer packages = fewer vulnerabilities).
- Regularly update base images.
- Remove unnecessary tools.

Use a minimal base image
FROM alpine:3.19
Install only what you need
RUN apk add --no-cache curl
Avoid apt/yum and extra packages

2. NON-ROOT EXECUTION



- Do not run containers as root.
- Use a non-root user.
- Set proper file permissions.
- Drop unnecessary privileges.
- Verify the user in the image.

Create a non-root user
RUN adduser -D appuser
USER appuser
Verify
whoami # should return appuser

3. LINUX CAPABILITIES



- Drop all unnecessary capabilities.
- Use the principle of least privilege.
- Add only required capabilities (e.g. NET_BIND_SERVICE).
- Avoid --privileged mode.
- Review capabilities for each service.

Drop all capabilities
docker run --cap-drop=ALL \\
--cap-add=NET_BIND_SERVICE \\
myapp:1.0
Check capabilities
capsh --print

4. SECRETS MANAGEMENT



- Do not store secrets in images.
- Use environment variables, secret managers or orchestration secrets (Kubernetes, Docker Swarm).
- Avoid hardcoded credentials.
- Rotate secrets regularly.
- Use least privilege access.

Example (Docker - env variable)
docker run -e DB_PASSWORD=***** \\
myapp:1.0
Example (Kubernetes)
apiVersion: v1
kind: Secret
metadata: { name: db-secret }

5. IMAGE SCANNING



- Scan images for vulnerabilities before deployment.
- Scan regularly (CI/CD pipeline).
- Fail builds on critical/high vulnerabilities.
- Use tools like Trivy, Gype, Snyk or Clair.
- Also scan base images.

Scan with Trivy
trivy image myapp:1.0
Scan with Snyk
snyk container test myapp:1.0
Scan in CI (example)
snyk container test --file=Dockerfile

6. IMAGE SIGNING & VERIFICATION



- Sign images to ensure integrity and authenticity.
- Use tools like Cosign or Notary.
- Verify signatures before deployment.
- Store signatures in a trusted registry.

Sign with Cosign
cosign sign myapp:1.0
Verify signature
cosign verify myapp:1.0
Use keyless signing (recommended)
cosign sign --keyless myapp:1.0

7. SBOMs (SOFTWARE BILL OF MATERIALS)



- Generate SBOMs for all images.
- Track dependencies and licenses.
- Use tools like Syft or Docker SBOM.
- Store SBOMs with the artifact.
- Use SBOMs for vulnerability management and compliance.

Generate SBOM with Syft
syft myapp:1.0 -o spdx-json > sbom.json
Docker SBOM
docker sbom myapp:1.0 > sbom.json

8. SUPPLY-CHAIN CONTROLS



- Use trusted base images and registries.
- Verify image provenance.
- Scan third-party dependencies.
- Use signed artifacts.
- Implement policy checks (admission controllers, OPA, Kyverno).
- Monitor for tampering.

Example: Verify provenance (Cosign)
cosign verify --certificate-identity \\
"https://github.com/yourorg" myapp:1.0
Use policy (Kyverno/OPA) to enforce signed images

9. RUNTIME SECURITY



- Use read-only root filesystem where possible.
- Set resource limits (CPU, memory).
- Use seccomp and AppArmor profiles.
- Restrict network access.
- Monitor container behaviour.
- Use runtime security tools (Falco, Aqua, etc.).

Read-only filesystem
docker run --read-only myapp:1.0
Set resource limits
docker run --memory=512m --cpus=0.5 \\
myapp:1.0

COMMON MISTAKES



- Using latest tag in production.
- Running containers as root.
- Hardcoding secrets in images.
- Skipping vulnerability scans.
- Ignoring image signing and SBOMs.
- Using oversized base images.
- No runtime security controls.

KEY TAKEAWAYS



- Smaller images, smaller risk.
- Run as non-root, with least privilege.
- Scan, sign, and verify everything.
- Manage secrets securely.
- Use SBOMs and supply-chain controls.
- Implement runtime security.
- Security is a continuous process.

REMEMBER



"A secure container today prevents a major incident tomorrow."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



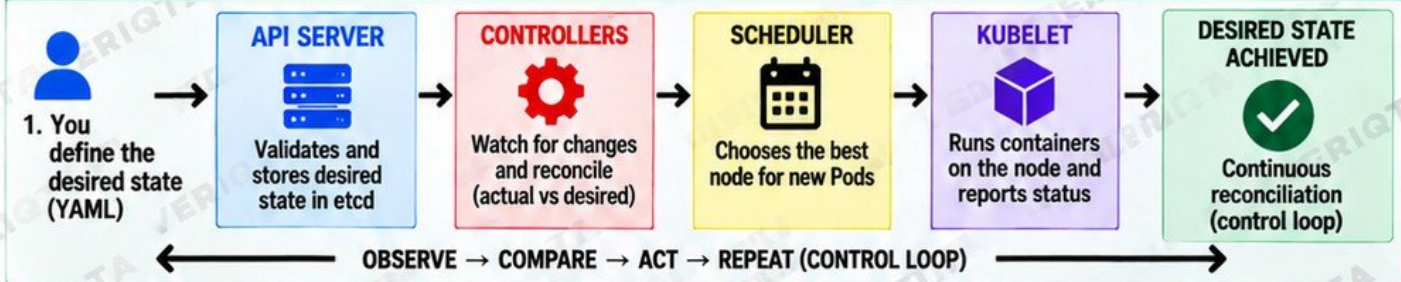
@veriqta

VERIQT
ENGINEER A
BETTER TOMORROW

Kubernetes Production Mental Model

API Server → Desired State → Controllers → Scheduler → Kubelet
Reconciliation and Control Loops

1. BIG PICTURE FLOW



2. KEY COMPONENTS



- **API Server:** Entry point. Validates and stores the desired state.
- **etcd:** Key-value store that holds the cluster state.
- **Controllers:** Continuously reconcile actual state to match desired state.
- **Scheduler:** Selects the best node for new Pods.
- **Kubelet:** Runs containers on the node and reports status to API server.
- **Kube-proxy:** Handles networking and service routing.
- **Container Runtime:** Runs the containers (e.g. containerd).

3. DESIRED STATE (DECLARATIVE)



- You tell Kubernetes what you want (e.g. Pods, Deployments, Services).
- Kubernetes continuously works to make it true.
- If something changes or fails, Kubernetes automatically fixes it (reconciliation).

Example: Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-app
spec:
 replicas: 3
 selector:
 matchLabels:
 app: my-app

4. RECONCILIATION LOOP



- Controllers watch the current state vs desired state.
- If there is a difference, they take action to fix it.
- This runs continuously (control loop).
- Handles failures, restarts, scaling, self-healing.

"Kubernetes constantly works to make the actual state match the desired state."

5. SCHEDULING



- Scheduler finds the best node for new Pods based on resources, constraints and policies.
- Considers: CPU, memory, node labels, taints/tolerations, affinity/anti-affinity, topology, and more.
- You can influence scheduling using nodeSelector, affinity, taints/tolerations.

```
# Example: nodeSelector
spec:
  template:
    spec:
      nodeSelector:
        disktype: ssd
```

6. KUBELET (ON EACH NODE)



- Receives pod specs from API server.
- Pulls container images.
- Runs containers using container runtime.
- Monitors container health.
- Reports status back to API server.
- Restarts containers if they fail.
- Follows the desired state (no manual work needed).

7. CONTROL LOOPS IN ACTION



- **Deployment controller:** keeps the right number of replicas running.
- **Node controller:** handles node failures.
- **Service controller:** keeps service endpoints updated.
- **Job/CronJob controller:** ensures jobs run to completion.
- **HPA controller:** automatically scales based on metrics (CPU, memory, custom metrics).

8. PRODUCTION MENTAL MODEL



- Define the desired state (YAML).
- Let controllers do the work.
- Trust the control loops.
- Design for failure (self-healing).
- Use observability (metrics, logs, events).
- Understand the flow: API server → controllers → scheduler → kubelet.
- Keep your manifests simple, clear and versioned.

COMMON MISTAKES



- Making manual changes on nodes.
- Not understanding the control loop.
- Hardcoding values instead of using ConfigMaps/Secrets.
- Ignoring resource requests and limits.
- Not using liveness and readiness probes.

KEY TAKEAWAY



"Kubernetes is not about running containers, it's about continually making the actual state match the desired state."

REMEMBER



"You declare, Kubernetes reconciles."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQT
ENGINEER A
BETTER TOMORROW

Kubernetes Pod Failure Triage

Pending, CrashLoopBackOff, OOMKilled, ImagePullBackOff, Probe Failures, Eviction, Events and Logs

1. POD LIFECYCLE & FAILURE POINTS



“Understand the normal flow, so you can quickly find what broke.”

2. PENDING



- Pod is created but not scheduled to a node.
- Common causes: insufficient resources, node selectors, taints, affinities, quota limits, PVC issues.

```
kubectl describe pod <pod>
kubectl get events --sort-by=.lastTimestamp
kubectl get nodes
kubectl describe pvc <pvc>
```

3. CRASHLOOPBACKOFF



- Container starts, crashes, and keeps restarting.
- Common causes: application errors, missing configuration, failed dependencies, bad startup command.

```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl logs <pod> --all-containers
```

4. OOMKILLED



- Container is killed due to out of memory.
- Common causes: memory limits too low, memory leaks, high traffic, inefficient application.

```
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl top pod <pod>
kubectl top node
kubectl get events | grep -i oom
```

5. IMAGEPULLBACKOFF



- Failed to pull the container image.
- Common causes: wrong image name or tag, private registry auth issues, network/DNS issues, rate limits.

```
kubectl describe pod <pod>
kubectl get events
kubectl get secret <image-pull-secret>
kubectl describe secret <secret>
```

6. PROBE FAILURES



- Liveness or readiness probe is failing.
- Common causes: application not ready, wrong probe configuration, slow startup, endpoint failures.

```
kubectl describe pod <pod>
kubectl get pod <pod> -o yaml
kubectl logs <pod>
kubectl exec -it <pod> -- curl -v http://localhost:<port>/<endpoint>
```

7. EVICTION



- Pod is evicted from the node.
- Common causes: node out of resources (memory, disk, PID), node maintenance, preemption (lower priority), node pressure.

```
kubectl get pods
kubectl describe pod <pod>
kubectl get events --field-selector involvedObject.name=<pod>
kubectl describe node <node>
```

8. EVENTS & LOGS



- Events show what happened in the cluster.
- Logs show why the container failed.
- Always check both.

```
kubectl get events --sort-by=.lastTimestamp
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl logs <pod> --all-containers
```

9. TRIAGE CHECKLIST



- Check pod status and events.
- Describe the pod for details.
- Check logs (current and previous).
- Verify resources (CPU, memory, disk).
- Check node conditions.
- Validate configuration (images, secrets, probes, etc.).
- Identify the root cause.
- Apply a fix and monitor.

10. KEY TAKEAWAYS



- Events are your first source of truth.
- Most pod issues are configuration or resource related.
- Check logs, don't guess.
- Understand probe behaviour.
- Monitor cluster resources.
- Fix the root cause, not just the symptom.
- Automate alerts for critical failures.

COMMON MISTAKES



- Only checking pod status (not events).
- Ignoring previous container logs.
- Not checking resource limits and node health.
- Assuming it's an application issue.
- Not verifying image names, tags or pull secrets.

REMEMBER



“Check events, check logs, find the cause, fix it, and prevent it from happening again.”

A STABLE CLUSTER
RUNS HAPPY WORKLOADS.



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Kubernetes Networking & DNS

Pod Networking, Services, EndpointSlices, CoreDNS, Ingress, NetworkPolicy, Service-Routing Failures

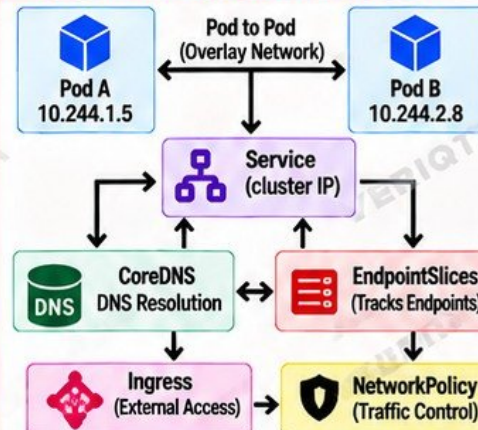
1. BIG PICTURE



- Pods get an IP address and can communicate with each other.
- Services provide stable access to Pods.
- EndpointSlices track ready endpoints.
- CoreDNS provides DNS resolution inside the cluster.
- Ingress exposes services to external traffic.
- NetworkPolicy controls traffic (flow of packets).

“Know how traffic flows, so you can find and fix issues faster.”

2. KUBERNETES NETWORKING MODEL



3. POD NETWORKING



- Each Pod gets a unique IP address.
- Pods can communicate across nodes (CNI plugin).
- No NAT between Pods.
- Ephemeral: Pod IP can change when recreated.

Check Pod IP
kubectl get pods -o wide
Test connectivity
kubectl exec -it <pod> -- ping <pod-ip>
Check CNI plugin
kubectl get pods -n kube-system

4. SERVICES



- Provides a stable virtual IP for a set of Pods.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.
- Load balances traffic to healthy Pods (via kube-proxy).
- Uses labels to select Pods.

List services
kubectl get svc
Describe service
kubectl describe svc <name>
Test service from inside cluster
kubectl exec -it <pod> -- curl http://<service-name>

5. ENDPOINTSlices



- Tracks IP addresses of ready Pods for a Service.
- More scalable than Endpoints (used in modern Kubernetes).
- Updated automatically as Pods are added/removed.

View EndpointSlices
kubectl get endpointslices
Describe EndpointSlice
kubectl describe endpointslice <name>
Check endpoints for a service
kubectl get endpointslice -l kubernetes.io/service-name=<svc>

6. COREDNS (DNS)



- Provides DNS resolution inside the cluster.
- Resolves service names to ClusterIP.
- Uses a Corefile for configuration.
- Runs as a Deployment in kube-system.

Check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns
Test DNS resolution
kubectl exec -it <pod> -- nslookup <service-name>
View CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns

7. INGRESS



- Manages external HTTP/HTTPS access to services.
- Routes based on hostnames or paths.
- Requires an Ingress controller (e.g., NGINX, Traefik).
- Uses rules to route traffic to services.

List ingresses
kubectl get ingress
Describe ingress
kubectl describe ingress <name>
Test ingress (with hostname)
curl -H "Host: example.com" http://<ingress-ip>

8. NETWORKPOLICY



- Controls traffic to and from Pods.
- Defines allow/deny rules based on labels, namespaces, IP ranges.
- Default: allow all (unless policy is applied).
- Helps with security and isolation.

List network policies
kubectl get networkpolicy
Describe policy
kubectl describe networkpolicy <name>
Example: deny all incoming traffic
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: deny-all
 selector: {}
policyTypes:
 - Ingress

9. COMMON SERVICE-ROUTING FAILURES



- DNS not resolving (CoreDNS issue).
- Service has no endpoints (Pods not ready).
- Wrong service type or port.
- NetworkPolicy blocking traffic.
- Ingress misconfiguration.
- CNI/network plugin issues.
- kube-proxy not working.
- Firewall or cloud security group blocking traffic.

10. TROUBLESHOOTING CHECKLIST



1. Check Pod status and IP.
2. Verify service and endpoints.
3. Test DNS resolution.
4. Check NetworkPolicy rules.
5. Verify ingress and controller logs.
6. Test connectivity (curl, ping, nslookup).
7. Check kube-proxy and CNI health.
8. Review events and logs.

KEY TAKEAWAYS



- Services + EndpointSlices = reliable routing.
- CoreDNS = service discovery.
- Ingress = external access.
- NetworkPolicy = traffic control.
- Most network issues are DNS, endpoints or policy related.

REMEMBER



“If it’s not resolvable, it’s not reachable.”



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Kubernetes Resources & Scaling

Requests, Limits, QoS, CPU Throttling, Memory Pressure, HPA, VPA, Cluster Autoscaler, Capacity

1. CONTAINER RESOURCES REQUESTS & LIMITS



- Requests: minimum resources guaranteed for the container.
- Limits: maximum resources the container can use.
- Kubernetes uses requests for scheduling and limits for enforcement.
- Set realistic values based on application behaviour.

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

2. QUALITY OF SERVICE (QoS)

Guaranteed

requests = limits
Most stable.
Best performance.

Burstable

requests < limits
Can use extra resources when available.

BestEffort

no requests, no limits
Lowest priority.
Can be evicted first.

"Set requests and limits to avoid noisy neighbors and ensure stability."

3. CPU THROTTLING



- Happens when a container exceeds its CPU limit.
- Kubernetes throttles the CPU to enforce the limit.
- Can cause slow application performance.
- Monitor with metrics (cpu.throttled).

Check CPU throttling
kubectl top pods

View detailed metrics (Prometheus)
rate(container_cpu_cfs_throttled_seconds_total[5m])

Describe pod for events
kubectl describe pod <pod-name>

4. MEMORY PRESSURE



- Occurs when node or container runs out of memory.
- Can cause OOMKilled (Out of Memory).
- Kubernetes may evict Pods under memory pressure.
- Set memory requests and limits based on real usage.

```
# Check memory usage
kubectl top pods
# Describe pod for OOM events
kubectl describe pod <pod-name>
# Check node memory pressure
kubectl describe node <node-name>
```

5. HORIZONTAL POD AUTOSCALER (HPA)



- Automatically scales the number of Pods based on metrics (CPU, memory, or custom).
- Helps handle traffic spikes.
- Uses metrics from Metrics Server or Prometheus.

```
# Create an HPA (CPU example)
kubectl autoscale deployment <name> \
  --cpu-percent=70 --min=2 --max=10
```

```
# View HPA
kubectl get hpa
# Describe HPA
kubectl describe hpa <name>
```

6. VERTICAL POD AUTOSCALER (VPA)



- Automatically adjusts CPU and memory requests (not limits).
- Helps right-size applications.
- Good for workloads with changing resource needs.
- Can run in recommendation, auto, or initial mode.

```
# Check VPA
kubectl get vpa
# View recommendations
kubectl describe vpa <name>
# Install VPA (using official manifests)
# https://github.com/kubernetes/autoscaler
```

7. CLUSTER AUTOSCALER



- Automatically adds or removes nodes based on pending Pods and resource demand.
- Helps maintain cluster capacity.
- Works with cloud providers (EKS, AKS, GKE, etc.).
- Reduces cost by scaling down unused nodes.

```
# Check cluster autoscaler status
kubectl -n kube-system logs deploy/cluster-autoscaler
# View pending pods (waiting for nodes)
kubectl get pods --all-namespaces \
  --field-selector=status.phase=Pending
```

8. CAPACITY PLANNING



- Monitor current and historical resource usage.
- Plan for growth and traffic spikes.
- Consider CPU, memory, storage, and node limits.
- Use monitoring tools (Prometheus, Grafana).
- Regularly review and adjust requests, limits, and autoscaling settings.

"Right-size today, scale for tomorrow."

9. USEFUL MONITORING COMMANDS



- Check resource usage:
kubectl top nodes
kubectl top pods
- View pod details:
kubectl describe pod <pod-name>
- Check events:
kubectl get events --sort-by=.lastTimestamp
- Monitor node resources:
kubectl describe node <node-name>

Monitor, alert, and scale before users feel the impact.

COMMON MISTAKES



- Not setting resource requests and limits.
- Using BestEffort Pods in production.
- Ignoring CPU throttling and memory pressure.
- Incorrect HPA/VPA configuration.
- Not monitoring cluster capacity.
- Reacting to issues instead of planning ahead.

KEY TAKEAWAYS



- Set realistic requests and limits.
- Understand QoS classes.
- Monitor and handle CPU/memory issues.
- Use HPA for scaling out.
- Use VPA for right-sizing.
- Use Cluster Autoscaler for node scaling.
- Plan capacity and monitor continuously.

REMEMBER



"Stable workloads today, scalable systems tomorrow."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Terraform Production Safety

State, Locking, Plans, Drift, Modules, Replacement Risk, Lifecycle, Imports, Secrets, Safe Applies

1. STATE MANAGEMENT



- State maps real infrastructure to your configuration.
- Keep state in remote backend (e.g. S3, Azure Storage, GCS).
- Never store state locally in production.
- State contains sensitive information.

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}
```

2. STATE LOCKING



- Prevents multiple people from running Terraform at the same time.
- Avoids state corruption.
- Use native locking (S3+DynamoDB, Azure, GCS) or Terraform Cloud.
- Always ensure locking is enabled in production.

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt       = true
  }
}
```

3. PLANS (REVIEW BEFORE APPLY)



- terraform plan shows what will change.
- Always review the plan output.
- Use -out to save the plan file.
- Have a peer review for production changes.

```
terraform plan -out=tfplan
```

```
# Review the plan
terraform show tfplan
```

```
# Apply the reviewed plan
terraform apply tfplan
```

4. DRIFT DETECTION



- Drift happens when real infrastructure is changed outside of Terraform (e.g. manual changes).
- Run terraform plan regularly to detect drift.
- Use monitoring and policy tools to prevent manual changes.

```
# Check for drift
terraform plan
```

```
# In CI/CD, run automated drift checks
```

5. MODULES (REUSABLE & SAFE)



- Use modules for consistency and reusability.
- Use versioned modules (e.g. from Terraform Registry or private repo).
- Keep modules simple and focused.
- Avoid hardcoding values; use variables.

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"
  name   = "prod-vpc"
  cidr   = "10.0.0.0/16"
}
```

6. REPLACEMENT RISK



- Some changes force resource replacement (e.g. instance type, disk size, subnet).
- Use terraform plan to identify resources marked for destroy/create.
- Understand the impact before apply.
- Use lifecycle rules to prevent accidental replacement.

```
resource "aws_instance" "app" {
  # ...
  lifecycle {
    prevent_destroy = true
  }
}
```

7. LIFECYCLE RULES



- Control how Terraform manages resources.
- Common rules: prevent_destroy, ignore_changes, create_before_destroy.
- Useful for critical resources.

```
resource "aws_db_instance" "db" {
  # ...
  lifecycle {
    prevent_destroy = true
    ignore_changes = ["tags"]
  }
}
```

8. IMPORTS (BRING EXISTING RESOURCES)



- Use terraform import to bring existing resources under Terraform management.
- Helps avoid recreation of existing infrastructure.
- After import, run terraform plan to verify the state.

```
# Import an existing resource
terraform import aws_vpc.main vpc-12345678

# Verify
terraform plan
```

9. SECRETS MANAGEMENT



- Never hardcode secrets in code.
- Use environment variables, secrets managers (AWS Secrets Manager, Azure Key Vault), or Terraform Cloud variables.
- Mark sensitive variables.

```
variable "db_password" {
  type      = string
  sensitive = true
}

# Use from environment variable
# TF_VAR_db_password=*****
```

10. SAFE APPLIES (PRODUCTION)



- Always review plan before apply.
- Use CI/CD with manual approval for production.
- Apply small, incremental changes.
- Use workspaces for environment separation.
- Have a rollback plan.
- Monitor after apply.

COMMON MISTAKES



- Storing state locally.
- No state locking.
- Applying without reviewing the plan.
- Ignoring drift.
- Hardcoding secrets.
- Accidental resource replacement.
- No approval process for production.

KEY TAKEAWAY



"Plan carefully, automate safely, and treat your infrastructure like production code."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQT
ENGINEER A
BETTER TOMORROW

Cloud Architecture Pocket Map

IAM, VPCs, Subnets, Routing, Load Balancing, Multi-AZ Design, Managed Services, Quotas and Fault Domains

1. CORE BUILDING BLOCKS



- Regions: physical locations.
- Availability Zones (AZs): isolated data centers within a region.
- VPC: your private network in the cloud.
- Subnets: split your network (public/private).
- Route Tables: control traffic flow.
- Internet Gateway (IGW): access to the internet.
- NAT Gateway: outbound internet for private subnets.

2. IAM (IDENTITY & ACCESS MANAGEMENT)



- Control who can access what.
- Use least privilege.
- Use roles instead of long-term access keys.
- Enable MFA.
- Use IAM policies, groups and permission boundaries.
- Audit with CloudTrail.

"Secure access is the foundation of a secure cloud."

3. VPC DESIGN (NETWORK LAYOUT)



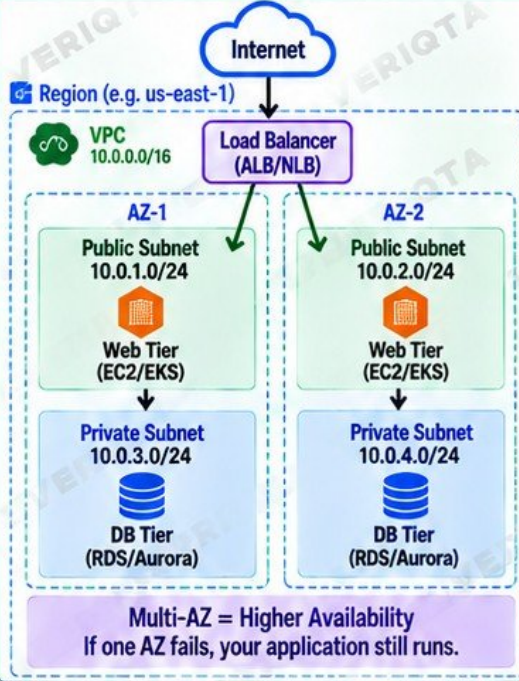
- Create a VPC with CIDR block (e.g. 10.0.0.0/16).
- Use public subnets (IGW) for internet-facing resources.
- Use private subnets (NAT) for internal resources.
- Use route tables to control traffic.
- Use security groups (instance firewalls).
- Use Network ACLs (subnet level).

4. LOAD BALANCING



- Distribute traffic across multiple instances/AZs.
- Types:
 - Application Load Balancer (HTTP/HTTPS)
 - Network Load Balancer (TCP/UDP)
 - Gateway Load Balancer (appliances)
- Improve availability and scalability.

5. MULTI-AZ ARCHITECTURE (HIGH AVAILABILITY)



6. MANAGED SERVICES



- Compute: EC2, ECS, EKS, Lambda
- Databases: RDS, Aurora, DynamoDB
- Storage: S3, EBS, EFS, FSx
- Networking: VPC, Route 53, CloudFront
- Messaging: SQS, SNS, MSK
- Monitoring: CloudWatch
- Security: IAM, KMS, Secrets Manager

"Use managed services to reduce operational work."

7. ROUTING



- Route tables control where traffic goes.
- Common routes:
 - 0.0.0.0/0 → Internet Gateway (public subnets)
 - 0.0.0.0/0 → NAT Gateway (private subnets)
 - VPC peering / Transit Gateway (for multi-VPC)
- Use Route 53 for DNS routing and health checks.

8. QUOTAS & LIMITS



- Each cloud has service quotas (e.g. EC2 instance limits).
- Check limits before deployment.
- Monitor usage to avoid hitting limits.
- Request increases if needed.
- Common quotas: vCPUs, IP addresses, load balancers, RDS instances, API limits.

"Plan for scale. Know your limits."

9. FAULT DOMAINS



- Region: multiple AZs, isolated from other regions.
- Availability Zone: failure of one AZ should not affect others.
- Use multi-AZ for critical workloads.
- Design for failure (use health checks, auto scaling, replication).
- Test your disaster recovery plan.

"Assume failure. Design for resilience."

10. BEST PRACTICES



- Use least privilege IAM.
- Use private subnets for databases.
- Enable encryption (at rest & in transit).
- Use multi-AZ for critical systems.
- Use managed services where possible.
- Implement monitoring and alerting.
- Tag your resources.
- Document your architecture.
- Review costs and quotas regularly.

"Secure. Scalable. Resilient. Cost-aware."

COMMON MISTAKES



- Using root account.
- Over-permissioned IAM policies.
- Exposing databases to the internet.
- No multi-AZ (single point of failure).
- Ignoring service quotas.
- Not enabling logging/monitoring.
- Poor network design (wrong routes).
- Not planning for disaster recovery.
- Unexpected costs (no budget alerts).

"Plan, design, and review before you deploy."

KEY TAKEAWAY



A well-designed cloud architecture is secure, highly available, scalable, and cost-efficient.

REMEMBER



"Design for the present, but plan for the future."

CLOUD PROVIDERS



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Observability That Actually Helps

Metrics, Logs, Traces, Events, RED, USE, Correlation, Cardinality, Telemetry Gaps and Actionable Signals

1. THE FOUR PILLARS



METRICS

Numeric data over time (e.g. CPU, latency, requests).



LOGS

Detailed records of what happened (e.g. errors, application logs).



TRACES

End-to-end request flow across services (e.g. distributed tracing).



EVENTS

Discrete occurrences (e.g. deploys, scaling, config changes).

"Use all four together for full visibility."

2. RED METHOD (FOR SERVICES)



- **R** – Rate (requests per second)
- **E** – Errors (error rate)
- **D** – Duration (latency / response time)

Best for: user-facing services and APIs.

"Is it working? Is it failing? Is it slow?"

3. USE METHOD (FOR RESOURCES)



- **U** – Utilization (how much is being used)
- **S** – Saturation (how busy is it)
- **E** – Errors (error rate)

Best for: infrastructure (CPU, memory, disk, network).

"Is it healthy? Is it overloaded? Is it failing?"

4. CORRELATION (CONNECT THE DOTS)



- Correlate metrics, logs and traces to find the real cause.
- Use common identifiers (e.g. request ID, trace ID).
- Align timestamps across systems.
- Jump from an alert → logs → trace → root cause.

"Correlation turns data into answers."

5. CARDINALITY (KEEP IT UNDER CONTROL)



- Cardinality = number of unique values (e.g. labels, tags).
- High cardinality can cause high storage costs and slow queries.
- Avoid unbounded labels (e.g. user_id, session_id).
- Use low-cardinality labels (e.g. service, environment).

"Useful data is specific, not infinite."

6. TELEMETRY GAPS



- Missing metrics (no visibility into key areas).
- Incomplete logs (not enough context).
- No traces (can't follow the request flow).
- Missing business metrics (e.g. user signups, purchases).
- Blind spots in infrastructure (e.g. network, database).

"If it's not instrumented, it doesn't exist."

7. ACTIONABLE SIGNALS



- Focus on signals, not noise.
- Set meaningful alerts (with thresholds and context).
- Use SLOs and error budgets.
- Alert on user impact, not just infrastructure metrics.
- Include runbooks and next steps with alerts.

"An alert should tell you what to do next."

8. PRACTICAL EXAMPLES

```
# Prometheus metrics (RED)
http_requests_total
http_request_duration_seconds
http_requests_errors_total

# Structured logs (with trace ID)
{
  "level": "error",
  "message": "Payment failed",
  "trace_id": "abc123",
  "user_id": "u789"
}

# Trace (OpenTelemetry)
TraceID: abc123
Span: payment-service
Duration: 320ms
Status: ERROR
```

9. TOOLS (EXAMPLES)



Metrics: Prometheus, Grafana, CloudWatch, Datadog



Logs: ELK/EFK, Loki, CloudWatch Logs, Datadog



Traces: Jaeger, Zipkin, AWS X-Ray, Datadog APM



Events: Kubernetes events, Cloud provider events, audit logs

10. KEY TAKEAWAYS



- ✓ Use metrics, logs, traces and events together.
- ✓ Apply RED for services and USE for resources.
- ✓ Keep cardinality under control.
- ✓ Correlate data to find root causes.
- ✓ Close telemetry gaps.
- ✓ Focus on actionable signals.
- ✓ Observability reduces MTTR and improves reliability.

"Better observability. Faster answers. Happier users."

COMMON MISTAKES



- ✗ Alerting on every small issue.
- ✗ High cardinality labels.
- ✗ Missing traces.
- ✗ No correlation between data sources.
- ✗ Ignoring business metrics.
- ✗ Poor alert thresholds.
- ✗ No runbooks.
- ✗ Collecting data but never using it.

"More data is not the goal. Better decisions are."

REMEMBER



"Observability is not about collecting data. It's about finding and fixing problems faster and preventing them in the future."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQT
ENGINEER A
BETTER TOMORROW

SRE Reliability Toolkit

SLIs, SLOs, Error Budgets, Burn Rates, Capacity, Graceful Degradation, Reliability versus Delivery Speed

1. SLI (SERVICE LEVEL INDICATOR)



- An SLI is a measurable indicator of how your service is performing.
- Examples: request success rate, latency, availability, throughput.
- Should be objective, accurate and user-focused.
- Use real user traffic (not just synthetic).

Common SLIs:

- Request success rate (%)
- Latency (p50, p95, p99)
- Availability (%)
- Throughput (requests/sec)

"If you can't measure it, you can't improve it."

2. SLO (SERVICE LEVEL OBJECTIVE)



- An SLO is a target for your SLI.
- It defines the acceptable level of performance.
- Example: 99.9% of requests succeed, or 95% under 200ms.
- Should be realistic and based on user needs and business requirements.

Example SLO:

- 99.9% availability (≤ 43 minutes downtime per month)
- 95% of requests ≤ 200 ms

"Set SLOs your team can meet and your users can trust."

3. ERROR BUDGET



- The error budget is the amount of allowable failure (100% - SLO).
- It gives you room to innovate while maintaining reliability.
- When the error budget is exhausted, stop risky changes and focus on stability.

Example (99.9% SLO):

- 0.1% error budget
- = 43 minutes of downtime per month

"The error budget is your license to move fast, responsibly."

4. BURN RATES



- Burn rate shows how quickly you are using your error budget.
- Helps detect issues early before you exhaust the budget.
- Common thresholds:
 - Fast burn: $14\times$ (in 1 hour)
 - Slow burn: $7\times$ (in 6 hours)

Alerting example:

- Page if burn rate $> 14\times$ (1h)
- Ticket if burn rate $> 7\times$ (6h)

"Measure burn rate, not just outages."

5. CAPACITY PLANNING



- Ensure your service can handle current and future demand.
- Consider CPU, memory, storage, network and dependencies.
- Use historical data and forecasting.
- Plan for peak traffic (campaigns, seasonal, regional).
- Include buffer capacity and auto-scaling where possible.

"Plan capacity before you need it, not during an incident."

6. GRACEFUL DEGRADATION



- When parts of the system fail, reduce functionality gracefully instead of total failure.
- Examples: serve cached data, limit features, return partial results, use fallback services.
- Keep core user journeys working even during incidents.
- Communicate clearly to users.

"Degradate gracefully, keep users productive."

7. RELIABILITY VS DELIVERY SPEED



- Faster delivery increases risk if not managed.
- Use the error budget to balance innovation and stability.
- High-risk changes should be limited when the error budget is low.
- Adopt progressive delivery (canary, feature flags, blue/green).
- Reliability and speed are not enemies — they complement each other.

"Move fast, but not at the cost of reliability."

8. PRACTICAL TOOLKIT



- ✓ Define SLIs for your critical services
- ✓ Set realistic SLOs
- ✓ Track error budgets and burn rates
- ✓ Monitor capacity and plan ahead
- ✓ Implement graceful degradation
- ✓ Use feature flags and progressive delivery
- ✓ Have clear incident response runbooks
- ✓ Review and adjust regularly

"SRE is a toolkit, not a one-time setup."

9. COMMON MISTAKES



- ✗ No clear SLIs or SLOs
- ✗ Setting unrealistic SLOs
- ✗ Ignoring the error budget
- ✗ Reacting late to burn rate alerts
- ✗ No capacity planning
- ✗ Total service failure instead of graceful degradation
- ✗ Prioritizing speed over reliability
- ✗ Not reviewing and updating SLOs

"Unclear goals, no data, and no plan lead to unreliable services."

10. KEY TAKEAWAYS



- ✓ SLIs measure. SLOs set the target.
- ✓ Error budgets enable innovation.
- ✓ Burn rates give early warning.
- ✓ Plan capacity and automate scaling.
- ✓ Design for graceful degradation.
- ✓ Balance reliability with delivery speed.
- ✓ Review, learn, and improve continuously.

REMEMBER



"Reliable systems earn trust, retain users, and enable growth."

REAL SRE IMPACT



- ✓ Happier users
- ✓ Fewer incidents
- ✓ Faster recovery
- ✓ Sustainable growth
- ✓ Stronger engineering culture



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Production Incident Response

Detect → Scope → Correlate → Hypothesize → Mitigate → Recover → Verify → Learn, Communication and Ownership

1. DETECT



- Alert from monitoring (metrics, logs, traces)
- User reports or support tickets
- Synthetic checks or external signals
- Anomaly detection

Examples:

- High error rate
- Latency spike
- Service down
- Increased 5xx responses

“Detect early, reduce impact.”

2. SCOPE



- Understand what is affected
- Identify impacted services, users and regions
- Determine severity (SEV)
- Set incident size and priority

Ask:

- Which services are impacted?
- How many users are affected?
- Is there data loss?
- Is this a regional or global issue?

“Know the blast radius.”

3. CORRELATE



- Bring data together (metrics, logs, traces, events)
- Find patterns and common timelines
- Identify related changes (deployments, config, infra)
- Look for dependencies

Tools:

- Grafana / Prometheus (metrics)
- Loki / ELK (logs)
- Jaeger / OpenTelemetry (traces)
- Kubernetes events, cloud logs

“One timeline, one story.”

4. HYPOTHESIZE



- Form the most likely cause(s)
- Use data, not guesswork
- Validate or disprove quickly
- Consider multiple hypotheses in parallel

Example hypotheses:

- Recent deployment caused issue
- Database is overloaded
- A dependency is failing
- Configuration change introduced error

“Make an educated guess, then prove it.”

5. MITIGATE



- Take immediate action to reduce user impact
- Use rollback, feature flags, scaling or traffic routing
- Apply workarounds if needed
- Keep communicating

Common actions:

- Rollback deployment
- Scale up resources
- Disable non-critical features
- Route traffic to healthy region
- Apply configuration fix

“Stop the bleeding.”

6. RECOVER



- Restore service to normal operation
- Verify all components are healthy
- Monitor closely for stability
- Remove temporary workarounds carefully

Checklist:

- Service is working as expected
- Error rates back to normal
- No lingering user impact
- All regions and dependencies healthy

“Recovery is not the end.”

7. VERIFY



- Confirm the issue is fully resolved
- Check metrics, logs and user experience
- Ensure no side effects
- Monitor for a sustained period

Verification steps:

- Error rate at baseline
- Latency normal
- User-facing functionality working
- No new related alerts

“Verify before you declare victory.”

8. LEARN



- Conduct a blameless postmortem
- Identify root cause(s)
- Document timeline, impact and what went well/poorly
- Define and track action items
- Update runbooks and alerts

Action items:

- Prevent recurrence
- Improve detection
- Improve response time
- Share learnings with the team

“Turn incidents into improvements.”

9. COMMUNICATION



- Keep stakeholders informed (internal and external)
- Be clear, honest and timely
- Use a single source of truth (e.g. status page, incident channel)
- Provide regular updates even if there is no new information

Communication tips:

- State what you know, what you don't know
- Give estimated next update time
- Avoid technical jargon for non-technical audiences

“Communicate early, communicate often.”

10. OWNERSHIP



- Have a clear incident owner
- Define roles and responsibilities
- Empower the team to make decisions
- Ensure follow-through on action items
- Foster a blameless culture

“Clear ownership drives faster resolution.”

COMMON MISTAKES



- Slow detection and response
- Not checking recent changes
- Jumping to conclusions
- Poor communication
- Declaring victory too early
- No follow-up or action items

“Learn from mistakes, don't repeat them.”

KEY TAKEAWAY



“A good incident response reduces impact today and builds a more reliable tomorrow.”



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Resilience & Failure Engineering

Timeouts, Retries, Exponential Backoff, Jitter, Circuit Breakers, Rate Limits, Bulkheads, Dependency Isolation

1. TIMEOUTS



- Set a maximum time to wait for a response.
- Prevents stuck requests and resource exhaustion.
- Use timeouts for all external calls (APIs, DBs, queues, etc.).
- Choose sensible values based on service behavior.

Example (Python):
`response = requests.get(url, timeout=5)`

“Fail fast, don’t wait forever.”

2. RETRIES



- Retry failed requests for transient errors (e.g. network, 5xx).
- Do not retry on permanent errors (e.g. 4xx).
- Use a limited number of retries.
- Combine with exponential backoff and jitter.

Example (pseudo):
`for i in range(3):
 try: call_service()
 except: retry()`

“Retry smart, not blindly.”

3. EXPONENTIAL BACKOFF



- Increase the wait time between retries exponentially.
- Reduces load on struggling services.
- Common formula:
 $\text{initial_delay} \times 2^{\text{attempt}}$
- Set a maximum backoff time to avoid very long waits.

Example:
Attempt 1: 1s
Attempt 2: 2s
Attempt 3: 4s
Attempt 4: 8s (max)

“Give systems time to recover.”

4. JITTER



- Add random variation to retry wait times.
- Prevents the “thundering herd” problem.
- Use full jitter or equal jitter strategies.
- Especially important in distributed systems.

Example (full jitter):
`wait_time = random(0, base_delay * 2^attempt)`

“Avoid synchronization, add jitter.”

5. CIRCUIT BREAKERS



- Stop calling a failing service after a threshold of failures.
- Allows the service time to recover.
- Has three states: Closed, Open, Half-Open.
- Use health checks to determine when to retry.

States:
Closed = normal operation
Open = fail fast
Half-Open = test if service recovered

“Fail fast, recover safely.”

6. RATE LIMITS



- Limit the number of requests to a service.
- Protects against abuse and accidental overload.
- Can be client-side or server-side.
- Return meaningful responses (e.g. 429 Too Many Requests).

Example (API response):
HTTP 429 Too Many Requests
Retry-After: 30

“Keep systems stable under load.”

7. BULKHEADS



- Isolate components so failures don’t spread.
- Use separate pools (threads, connections, queues).
- Prevent one failing service from consuming all resources.
- Apply at the service, thread, or infrastructure level.

Example:
Separate DB connection pool for critical and non-critical services.

“Contain failures, protect the core.”

8. DEPENDENCY ISOLATION



- Minimize blast radius of external dependencies.
- Use queues, async processing, and caching.
- Provide fallbacks or degraded responses.
- Design for partial failures (e.g. optional features).

Patterns:
• Async processing
• Fallback responses
• Caching
• Feature flags

“Design for failure, not perfection.”

9. REAL-WORLD EXAMPLE



E-COMMERCE CHECKOUT FLOW

- Timeout: 2s for payment API
- Retry: 3 attempts with backoff
- Jitter: randomize wait time
- Circuit breaker: open after 5 failures
- Rate limit: 100 requests/min per user
- Bulkhead: separate pool for payments
- Dependency isolation: use queue and fallback to “order pending”

“Combine patterns for true resilience.”

10. KEY TAKEAWAYS



- Set sensible timeouts.
- Use retries with exponential backoff and jitter.
- Implement circuit breakers.
- Apply rate limits.
- Use bulkheads and isolate dependencies.
- Test failure scenarios regularly.
- Resilience is a feature, not an afterthought.

“Build systems that survive the bad days.”

COMMON MISTAKES



- No timeouts (requests hang).
- Unlimited retries (makes things worse).
- No jitter (thundering herd).
- Not using circuit breakers.
- Ignoring rate limits.
- Sharing resources between critical and non-critical services.
- No fallback or degraded mode.

“Hope is not a strategy.”

REMEMBER



“Resilient systems are not lucky. They are designed, tested and continuously improved.”



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW

Senior DevOps Production Checklist

Pre-deployment, Security, Observability, Reliability, Capacity, Rollback, DR, Ownership and Operational-Readiness Checks

1. PRE-DEPLOYMENT CHECKS



- ☐ Requirements are clear and approved
- ☐ Code reviewed and tested
- ☐ All tests (unit, integration, e2e) passing
- ☐ Build artifacts verified (version/tag)
- ☐ Configuration changes reviewed
- ☐ Feature flags in place (if needed)
- ☐ Deployment plan and rollback plan ready
- ☐ Change window and stakeholders informed

"Plan it. Test it. Then ship it."

2. SECURITY CHECKS



- ☐ Dependencies scanned (SCA)
- ☐ Container images scanned
- ☐ No hardcoded secrets
- ☐ Secrets stored in vault/secret manager
- ☐ Least privilege access (IAM/RBAC)
- ☐ Network policies / firewalls reviewed
- ☐ Security headers and TLS enabled
- ☐ Compliance requirements met (e.g. PCI, HIPAA, SOC 2)

"Security is a feature, not an afterthought."

3. OBSERVABILITY CHECKS



- ☐ Metrics are collected and visible
- ☐ Logs are centralized and searchable
- ☐ Traces enabled (distributed tracing)
- ☐ Dashboards updated and validated
- ☐ Alerts configured (SLO/SLI based)
- ☐ Key business metrics tracked
- ☐ Runbooks and dashboards accessible
- ☐ Synthetic/real user monitoring enabled

"If you can't see it, you can't run it."

4. RELIABILITY CHECKS



- ☐ SLIs and SLOs defined
- ☐ Error budgets configured
- ☐ Retries, timeouts and circuit breakers implemented
- ☐ Graceful degradation tested
- ☐ Dependency failures handled
- ☐ Service health checks (readiness/liveness)
- ☐ Chaos/failure testing (if possible)
- ☐ Known failure scenarios documented

"Build for failure, not for perfection."

5. CAPACITY CHECKS



- ☐ Resource requests and limits set
- ☐ HPA/VPA configured (if applicable)
- ☐ Cluster/node capacity verified
- ☐ Database capacity and connections checked
- ☐ Storage capacity and IOPS reviewed
- ☐ Expected traffic load tested
- ☐ Autoscaling rules validated
- ☐ Cost impact reviewed

"Scale with data, not assumptions."

6. ROLLBACK CHECKS



- ☐ Rollback plan documented and tested
- ☐ Previous stable version available
- ☐ Database rollback strategy defined
- ☐ Configuration rollback (e.g. IaC) ready
- ☐ Feature flags can disable new changes
- ☐ Rollout strategy (blue/green, canary) validated
- ☐ Rollback time and impact understood
- ☐ Team is familiar with the process

"Always have a way back."

7. DISASTER RECOVERY CHECKS



- ☐ Backups are working and tested
- ☐ Recovery time objective (RTO) defined
- ☐ Recovery point objective (RPO) defined
- ☐ Multi-AZ / multi-region setup validated
- ☐ Failover process tested (runbook)
- ☐ Critical data replication verified
- ☐ Dependencies (DNS, third-party) included
- ☐ DR contacts and escalation plan ready

"Hope for the best. Prepare for the worst."

8. OWNERSHIP CHECKS



- ☐ Service owner(s) clearly defined
- ☐ On-call team identified
- ☐ Escalation path documented
- ☐ Runbooks and SOPs up to date
- ☐ Knowledge transfer completed
- ☐ Dependencies and stakeholders listed
- ☐ Team has access to required tools
- ☐ Ownership recorded (e.g. in catalog)

"Clear ownership drives faster resolution."

9. OPERATIONAL-READINESS CHECKS



- ☐ Monitoring and alerts validated
- ☐ Runbooks accessible and tested
- ☐ Incident response plan in place
- ☐ On-call team briefed
- ☐ Communication plan ready (internal and external)
- ☐ Support teams aware (e.g. NOC, Helpdesk)
- ☐ Post-deployment validation steps defined
- ☐ Go/No-Go approval obtained

"Ready to operate, not just deploy."

10. KEY TAKEAWAYS



- ✓ Use this checklist for every production change.
- ✓ Automate as many checks as possible.
- ✓ Keep it updated for your environment.
- ✓ Involve the right people early.
- ✓ A safe deployment is a team effort.
- ✓ Reliability, security and observability enable business success.

"Check. Validate. Deploy. Operate. Improve."

COMMON MISTAKES



- ✗ Skipping tests or reviews
- ✗ Ignoring security checks
- ✗ No rollback plan
- ✗ Insufficient monitoring
- ✗ Underestimating capacity
- ✗ Unclear ownership
- ✗ No DR testing
- ✗ Poor communication
- ✗ Rushing to production

"Small misses can become big incidents."

REMEMBER



- ✓ Production is a privilege.
- ✓ Users trust us to keep it running.
- ✓ Reliability, security and scalability create real value.
- ✓ Operational excellence is a habit.
- ✓ A great DevOps engineer thinks beyond deployment.

"Do it right. Keep it right."



@veriqta



/veriqta



/veriqta



/veriqta



/veriqta



@veriqta

VERIQTA
ENGINEER A
BETTER TOMORROW